

这部分com域和uds域依赖库是怎样的，平台库和sdk库以及协议栈库帮我理一下

- Thought for 3s >
- Read consys\_bf.cmake
- Thought for 2s >
- Read microsar\_config\_com.cmake
- Thought for 1s >
- Read microsar\_config\_uds.cmake
- Thought for 2s >
- Read CMakeLists.txt
- Thought for 1s >
- Read CMakeLists.txt
- Thought for 20s >
- 现在清楚了。以下是 COM 域和 UDS 域的完整依赖库拆解。

COM 域 (libdji\_com\_service.so / fawhq\_e001\_10\_com.so)

链接命令 (consys\_bf.cmake:213):

```
target_link_libraries(${PRODUCT_NAME}_com PRIVATE
-Wl,--whole-archive
    ${product_static_lib_list}          # ①+②+③+④ 打包
    autosar_com_${PRODUCT_NAME}        # ⑤
    oem_proxy_${PRODUCT_NAME}          # ⑥
-Wl,--no-whole-archive
)
```

库清单

| 分类            | 库名                     | 代码位置                            |
|---------------|------------------------|---------------------------------|
| 【BSP层】        |                        |                                 |
| ①             | dcos_dcms              | DCMS IPC实现 Qualcomm BSP提供       |
|               | mini_dcos              | mini DCOS(仿真用) Qualcomm BSP提供   |
|               | \${DCOS_LIBRARIES}     | 其他BSP库 Qualcomm BSP提供           |
| 【平台层 SDK】     |                        |                                 |
| ②             | plat_bf_cdd.so         | comif运行时引擎 plat-bf/dsar_app/    |
|               | (DSAR_APP_LIB_CDD)     | cdd/                            |
|               | service_route.cpp      | SOME/IP路由引擎                     |
|               | x_dom_someip_rt.cpp    | SOME/IP入口                       |
|               | msg_route_soc.cpp      | CAN消息路由引擎                       |
|               | sig_route_soc.cpp      | CAN信号路由                         |
|               | x_dom_can_rt_soc.cpp   | CAN comif入口                     |
|               | dcms_adapt/            | DCMS适配封装                        |
| ③             | APP_CORE_LIB           | app_core业务框架 dsar_app/app_core/ |
| 【车型适配层 proxy】 |                        |                                 |
| ④             | x_dom_can_rt_gen_lib   | CAN路由表+信号 proxy/x_dom_can_rt_   |
|               | com_\${PRODUCT}.a      | 定义(SigHdl, gen_com/             |
|               |                        | CanMsg, map)                    |
|               | x_dom_someip_rt_gen    | SOME/IP路由表 proxy/x_dom_someip_  |
|               | _lib_com_\${PRODUCT}.a | (svc_route_recv_ rt_gen_com/    |
|               |                        | svc_info_map,                   |
|               |                        | NodeRecvInfo,                   |
|               |                        | SvcItemInfo)                    |

|                                |                        |                       |  |
|--------------------------------|------------------------|-----------------------|--|
| 【AUTOSAR 协议栈】                  |                        |                       |  |
| ⑤ autosar_com                  | AUTOSAR COM BSW        | dsar-sip/sip_autosar/ |  |
| \${PRODUCT}_a                  | SoAd                   | 以太网socket             |  |
|                                | SomeIpTp               | SOME/IP协议解析 -关键!      |  |
|                                | Sd                     | Service Discovery     |  |
|                                | LdCom                  | 信号分解                  |  |
|                                | Com                    | 信号映射                  |  |
|                                | PduR                   | PDU路由                 |  |
|                                | Cdd_SomeIpTp.c         | - 车型自己的               |  |
|                                | IpduM/BswM/ComM/Nm/... |                       |  |
|                                | E2E/Crc                |                       |  |
| 依赖: plat_bf_cdd.so + mini_dcos |                        |                       |  |
|                                |                        |                       |  |
|                                |                        |                       |  |
| 【OEM proxy】                    |                        |                       |  |
| ⑥ oem_proxy_\${PRODUCT}        | OEM传感器代理               | proxy/SensorGroup/    |  |
|                                |                        |                       |  |
|                                |                        |                       |  |

## UDS 域 (libuds\_v4.so)

链接命令 (consys\_of\_cmake:237-238):

```
target_link_libraries(uds_v4 PRIVATE
-Wl,--whole-archive
  ${product_static_lib_list_uds_v4} # ①+②+③+④ 打包
  autosar_uds_${PRODUCT_NAME}_v4 # ⑤
-Wl,--no-whole-archive
)
target_link_libraries(uds_v4 PRIVATE ${DCOS_LIBRARIES} mini_dcos)
```

### 库清单

|                                            |                    |                        |  |
|--------------------------------------------|--------------------|------------------------|--|
|                                            |                    |                        |  |
| 【BSP层】                                     |                    |                        |  |
| dcos_dcms + mini_dcos + \${DCOS_LIBRARIES} |                    |                        |  |
| → 和 COM 域同一套 BSP 库                         |                    |                        |  |
|                                            |                    |                        |  |
| 【平台层 SDK】                                  |                    |                        |  |
| ② plat_bf_cdd.so                           | comif运行时引擎         | - 和COM同一份.so文件!        |  |
| 同一份代码,但运行时根据 g_x_dom_someip_config 走不同分支:  |                    |                        |  |
| - com_service_enabled = false              |                    |                        |  |
| - app_service_enabled = true               |                    |                        |  |
| - svc_route_com_to_app_topic_cbk()         | 负责收DCMS数据          |                        |  |
|                                            |                    |                        |  |
| ③ UDSONIP_TARGET_NAME_v4                   | UDS over IP封装      |                        |  |
| \${DSAR_APP_LIB_UDSONIP}                   |                    |                        |  |
|                                            |                    |                        |  |
| 【车型适配层 proxy】                              |                    |                        |  |
| ④ x_dom_can_rt_gen_lib                     | CAN信号缓存            | proxy/x_dom_can_rt_    |  |
| uds_\${PRODUCT}_v4.a                       | (几乎为空)             | gen_uds/               |  |
|                                            |                    |                        |  |
| x_dom_someip_rt_gen                        | SvcItemInfo定义      | proxy/x_dom_someip_    |  |
| lib_uds_\${PRODUCT}                        | SigIf Get/Set      | rt_gen_uds/            |  |
| _v4.a                                      | 序列化/反序列化           |                        |  |
|                                            |                    |                        |  |
| diagproxy_ge                               | C-C++桥接            | DiagProxy/             |  |
| \${PRODUCT}_a                              | (诊断服务调             |                        |  |
|                                            | comif SigIf)       |                        |  |
|                                            |                    |                        |  |
| 【AUTOSAR 协议栈】                              |                    |                        |  |
| ⑤ autosar_uds                              | AUTOSAR UDS BSW    | dsar-sip + oem_feature |  |
| \${PRODUCT}_v4.a                           | DoIP               | 诊断仪IP通信 (端口13400)      |  |
|                                            | Dcm                | 诊断通信管理                 |  |
|                                            | Dem                | 诊断事件管理                 |  |
|                                            | PduR               | PDU路由                  |  |
|                                            | SoAd               | 以太网socket - 只配了DoIP端口! |  |
|                                            | Fee/Fls/NvM        | 存储                     |  |
|                                            | BswM/ComM          | 模式管理                   |  |
|                                            | DagnosticService.c | 车型适配诊断逻辑               |  |
|                                            | DiagSwc_Callback.c | DID读与回调                |  |





两者是独立进程，通过 DCMS (BSP提供的IPC) 通信

## 1.1 COM域库依赖

fawhq\_e001\_10\_com.so

```

└─ BSP层
    ├── dcms_dcms          DCMS IPC实现 (Qualcomm BSP)
    ├── mini_dcos          mini DCOS (仿真)
    ├── ${DCOS_LIBRARIES}  其他BSP库
    ├── ${CONAN_LIBS}      Conan包
    └─ 平台仓SDK (plat-bf)
        ├── plat_bf_cdd.so    共享库, 包含:
        │   ├── service_route.cpp    ← SOME/IP路由引擎
        │   ├── x_dom_someip_rt.cpp  ← SOME/IP comif入口
        │   ├── msg_route_soc.cpp    ← CAN消息路由引擎
        │   ├── sig_route_soc.cpp    ← CAN信号路由
        │   ├── x_dom_can_rt_soc.cpp ← CAN comif入口
        │   ├── dcms_adapt/          ← DCMS API封装
        │   └── x_dom_someip_rt_com.h ← SvcItemInfo/SvcDataPackHdl类定义
        ├── APP_CORE_LIB      app_core业务框架库
        └─ 车型适配仓 proxy (静态库, --whole-archive)
            ├── x_dom_can_rt_gen_lib_com {PRODUCT}.a
            │   ├── x_dom_can_rt_cfg.cpp ← CanMsg对象(37个), SigHdl对象(60+),
            │   │   │   msg_route_mcu_tx_rx_msgs_map,
            │   │   │   sig_route_send_mcu_tx_rx_sigs()
            │   └── x_dom_can_rt_sigif.cpp ← SigIf_Get_xxx() 信号读取接口
            ├── x_dom_someip_rt_gen_lib_com {PRODUCT}.a
            │   ├── x_dom_someip_rt_cfg.cpp ← SvcItemInfo(120+个),
            │   │   │   svc_route_rcv_svc_info_map,
            │   │   │   svc_route_send_svc_info_map,
            │   │   │   NodeRecvInfo(uds_app + ad_app列
            │   │   │   svc_route_rcv_svc_info_list
            │   ├── x_dom_someip_rt_sigif_get.cpp ← SigIf_xxx_Get()
            │   ├── x_dom_someip_rt_sigif_set.cpp ← SigIf_xxx_Set()
            │   └── x_dom_someip_rt_protocol_*.cpp ← 序列化/反序列化函数
            ├── oem_proxy_{PRODUCT}.a    OEM传感器代理
            └─ AUTOSAR协议栈 (静态库, --whole-archive)
                ├── autosar_com {PRODUCT}.a
                │   ├── SoAd              ← 以太网Socket适配 (SOME/IP端口)
                │   ├── SomeIpTp          ← * SOME/IP协议解析 (只有COM有!)
                │   ├── Sd                ← * Service Discovery (只有COM有!)
                │   ├── LdCom             ← 信号分解
                │   ├── Com               ← 信号映射
                │   ├── PduR              ← PDU路由
                │   ├── Cdd_SomeIpTp.c    ← * 关键文件: SOME/IP回调-comif分发
                │   ├── Cdd_SomeIpTp.h    ← 类型定义
                │   ├── IpduM, BswM, ComM, Nm, UdpNm
                │   └── E2E, Crc
                └─ OEM Feature
                    ├── oem_feature {PRODUCT}.a    OEM功能特性

```

## 1.2 UDS域库依赖

libuds v4.so

└ BSP层

```

dcos_dcms + mini_dcos + ${DCOS_LIBRARIES}
(和COM域同一套BSP库)

平台仓SDK (plat_bf)
plat_bf_cdd.so * 和COM域同一份.so文件!
(运行时根据 g_x_dom_someip_config 走UDS分支)

${DSAR_APP_LIB_UDSONIP} UDS over IP封装

车型适配仓 proxy (静态库, --whole-archive)
x_dom_can_rt_gen_lib_uds {PRODUCT} v4.a
└─ x_dom_can_rt_cfg.cpp - CAN信号缓存结构体
    sig_route_update_mcu_tx_rx_sigs_from_com()
    (fawhq_e001_10: 几乎为空, CAN_UDS_V4=0FF)

x_dom_someip_rt_gen_lib_uds {PRODUCT} v4.a
└─ x_dom_someip_rt_cfg.cpp - SvcItemInfo(3 recv + 4 send),
    svc_route_recv_svc_info_map,
    svc_route_send_svc_info_map,
    svc_route_write_someip_init()
└─ x_dom_someip_rt_sigif_get.cpp - SigIf_OnRecvData(), SigIf_xxx_Get()
└─ x_dom_someip_rt_sigif_set.cpp - SigIf_xxx_Set()
└─ x_dom_someip_rt_protocol_0.cpp - Deserialize/Serialize

diagproxy_ge {PRODUCT}.a
└─ DiagProxy_Com.cpp - C-C++桥接, 诊断服务调comif SigIf

AUTOSAR协议栈 (静态库, --whole-archive)
autosar_uds {PRODUCT} v4.a
└─ DoIP - * 诊断仪IP通信 (端口13400, 只有UDS有!)
└─ Dcm - * 诊断通信管理
└─ Dem - 诊断事件管理
└─ PduR - PDU路由
└─ SoAd - 以太网Socket (只配DoIP端口!)
└─ Fee, Fls, Nvm - 存储
└─ BswM, ComM - 模式管理
└─ DiagnosticService.c - 车型适配诊断逻辑
└─ DiagSwc_Callback.c - DID读写回调
└─ Dcm_Callout_Stubs.c - Dcm钩子函数
└─ dji_rte.c - RTE接口注册

* 被注释掉的: Sd, SomeIpTp, LdCom, Com
* UDS无法从网络收SOME/IP!

OEM Feature
oem_feature {PRODUCT}_uds_v4.a
└─ security_cert_manager/ 安全证书管理
└─ DiagCalib/ 诊断标定
└─ FactoryParam/ 工厂参数
└─ SecurityAccess/ 安全访问
└─ DcmApp_Reset/ 应用复位
└─ UpdateAdapter/ 升级适配
└─ SubscribeProcess/ 订阅处理

其他
dsar_app_lib_udsonip UDS over IP库

```

## 二、Cdd\_SomeIpTp.c — SOME/IP 分流的“枢纽”

这是理解有无 comif 区别的最关键文件。

### 2.1 文件来源

SIP提供: dsar-sip/dsar\_app/sip\_autosar/xxx/BSW/SomeIpTp/\_Cdd\_SomeIpTp.c ~~→ 删除!~~  
 车型替换: autosar\_adapter/microsar\_config\_com/Appl/GenData/Cdd\_SomeIpTp.c ~~→ 使用这个~~

```

microsar_config_com.cmake:31-32:
list(REMOVE_ITEM COM_AUTOSAR_BSW_SRC_LIST .../ _Cdd_SomeIpTp.c
.../ _Cdd_SomeIpTp.h)

```

车型自己有 Cdd\_SomeIpTp.c, 由 JSON 配置工具自动生成, 替换 SIP 提供的版本。



## 2.2 文件统计

总 LdCom 回调函数: 846 个  
└─ 走 comif 路径: 114 个 → 调用 x\_dom\_someip\_rt\_rx\_handle()  
└─ 不走 comif: 732 个 → 只有 SOMEIP\_DEBUG\_INFO, 数据进 LdCom

## 2.3 两种路径的代码对比

```
// =====  
// 路径A: comif 路径 (114个服务, 需要跨域转发)  
// =====  
/*  
 * Service Name: Sf_SA_FDD_SeatOccupyInfo - 服务名  
 * Service Id: 0x6992 - SOME/IP Service ID  
 * Service Interface Element Name: StbltandArbg_NotifyCycleSeatBeltSt  
 * Service Interface Element Id: 0x9002 - Method/Event ID  
 * Client Server: Client  
 * Message Type: Event - Notification  
 * Transport Procol: UDP  
 * Event Group : EG_Sf_SA_FDD_SeatOccupyInfo@0x7001  
 */  
FUNC(void, RTE_CODE)  
LdComRxIndication_Client_Sf_SA_FDD_SeatOccupyInfo_StbltandArbg_NotifyCycleSeatBeltSt_Event_RX(  
    P2CONST(PduInfoType, AUTOMATIC, RTE_APPL_DATA) PduInfoPtr)  
{  
    uint8 *Data = (uint8 *)PduInfoPtr->SduDataPtr;  
    uint32 length = PduInfoPtr->SduLength;  
    SOMEIP_DEBUG_INFO("...SeatBeltSt_Event_id = 0x69929002 \r\n");  
  
    #ifdef X_DOM_SOMEIP_RT_COM_EN // - 编译宏控制  
        // * 直接把SOME/IP payload丢给comif SDK  
        x_dom_someip_rt_rx_handle(  
            X_DOM_SOMEIP_RT_SERVER_ID_Sf_SA_FDD_SeatOccupyInfo ... RX,  
            Data + CDD_SOMEIPTP_XF_DATA_OFFSET, // 跳过8字节SOME/IP头  
            PduInfoPtr->SduLength - CDD_SOMEIPTP_XF_DATA_OFFSET  
        );  
    #endif  
}  
// - 数据不进 LdCom/Com, 直接进 comif → DCMS → UDS/AD
```

```
// =====  
// 路径B: danvince 路径 (732个服务, COM进程自己消费)  
// =====  
/*  
 * Service Name: Veh_VM_PowerModeInfo  
 * Service Id: 0x900A  
 * Service Interface Element Name: PMM_NotifyONChgIGst  
 * Service Interface Element Id: 0x8003  
 * ...  
 */  
FUNC(void, RTE_CODE)  
LdComRxIndication_Client_Veh_VM_PowerModeInfo_PMM_NotifyONChgIGst_FieldNotify_RX(  
    P2CONST(PduInfoType, AUTOMATIC, RTE_APPL_DATA) PduInfoPtr)  
{  
    uint8 *Data = (uint8 *)PduInfoPtr->SduDataPtr;  
    uint32 length = PduInfoPtr->SduLength;  
    SOMEIP_DEBUG_INFO("...IGst_FieldNotify_id = 0x900a8003 \r\n");  
    // * 函数体为空! 数据由 AUTOSAR LdCom-Com 在COM进程内消费  
    // 不进comif, 不走DCMS  
}
```

## 2.4 谁决定哪些服务走comif?

JSON配置文件 (车型适配)  
└─ 代码生成工具  
└─ 生成 Cdd\_SomeIpTp.c - 决定 LdCom 回调里是否调用 x\_dom\_someip\_rt\_rx\_handle()



```

PS_FM_OTASelfDL_Mgmt_HAD_J0_OTASelfDL_RRSelfDownloadCmd_HAD_J0(
    SERVER_ID...RX, // - 与Cdd_SomeIpTp.c中的ID对应
    "PS_FM_OTASelfDL...",
    100, // 超时时间(ms)
    SigIf_OnRecvData, // - * 回调函数指针(存到m_recv_func)
    0); // 初始值

```

### Step 3: comif SDK 周期打包发送 (COM进程, 每50ms)

**[SDK] svc\_route\_cycle\_pub() [50ms定时任务]**

文件: plat-bf/cdd/x\_dom\_someip\_rt/service\_route.cpp

```

if (g_x_dom_someip_config.com_service_enabled)
{
    // * 遍历proxy定义的NodeRecvInfo列表
    for (auto& elem : *g_x_dom_someip_config
        .svc_route_rcv_svc_info_list_ptr)
    {
        check_and_send_svc_list_via_dcms_on_timeout(
            map_ptr,
            elem->svc_list, // 该Node应接收的服务ID列表
            elem->topic_name.c_str() // DCMS topic
        );
    }
}

```

内部逻辑:

```

├─ 遍历 svc_list 中的每个 service_id
├─ 从 map 中找到 SvcItemInfo
├─ 检查 is_timeout() && has_new_data()
├─ SvcItemInfo::get_origin_data() 获取原始SOME/IP payload
├─ * SvcDataPackHdl::add_data(id, data, len, rcv_count)
├─ 将多个服务的数据打包成一个二进制blob
├─ * dcms_mcu_topic_send(topic_id, packed_blob, blob_len)
└─ 通过DCMS发送

```

**[Proxy] svc\_route\_rcv\_svc\_info\_list (数据来源)**

文件: proxy/x\_dom\_someip\_rt/gen\_com/x\_dom\_someip\_rt\_cfg.cpp:430-444

// UDS应接收的服务:

```

static NodeRecvInfo svc_route_rcv_svc_info_uds_app = {
    {
        SERVER_ID...RRSelfDownloadCmd_HAD_J0_RX,
        SERVER_ID...RRSelfDownloadStatus_HAD_J0_RX,
        SERVER_ID...RRSelfDownloadScript_HAD_J0_RX,
    },
    "/x_dom_someip_rt/com_to_app/uds_app" // - DCMS topic
};

```

// AD应接收的服务 (120+个):

```

static NodeRecvInfo svc_route_rcv_svc_info_ad_app = {
    {SERVER_ID xxx_RX, SERVER_ID yyy_RX, ... },
    "/x_dom_someip_rt/com_to_app/ad_app"
};

```

### Step 4: DCMS跨进程传输

COM进程 —DCMS—> UDS进程

topic: "/x\_dom\_someip\_rt/com\_to\_app/uds\_app"  
payload: SvcDataPackHdl打包的二进制blob

### Step 5: UDS侧 comif SDK 接收解包 (UDS进程)

\* 初始化时(UDS proxy的 svc\_route\_write\_someip\_init):  
[Proxy] 注册DCMS回调



```
【Proxy】注册DCMS回调  
文件: proxy/x_dom_someip_rt_gen_uds/x_dom_someip_rt_cfg.cpp:64-77  
  
topic_id = dcms_adp_get_topic_id(  
    XDOMSOMEIP_RT_TOPIC_NODE_NAME,  
    XDOMSOMEIPRT_COM_TO_APP_TOPIC_NAME // = "/com_to_app/uds_app"  
);  
dcms_mcu_topic_setup_callback(topic_id, cbk, ...);  
// cbk = svc_route_com_to_app_topic_cbk
```

★ 收到DCMS数据时:

```
【SDK】svc_route_com_to_app_topic_cbk(data, len)  
文件: plat-bf/cdd/x_dom_someip_rt/service_route.cpp:147  
  
SvcDataUnPackHdl DataUnPackHdl(data, len); // SDK解包器  
  
while (1)  
{  
    // ★ SDK逐条解包  
    ret = DataUnPackHdl.get_next_data(  
        item_id, item_data, item_len, com_rcv_count);  
    if (!ret) break;  
  
    // ★ SDK查proxy的rcv map  
    auto item_it = g_x_dom_someip_config  
        .svc_route_rcv_svc_info_map_ptr->find(item_id);  
  
    item_ptr = item_it->second;  
    item_ptr->set_com_rcv_count(com_rcv_count);  
  
    // ★ SDK写入数据到SvcItemInfo  
    item_ptr->set_origin_data(item_id, item_data, item_len);  
}
```

```
【Proxy】svc_route_rcv_svc_info_map (UDS侧, 3个OTA服务)  
文件: proxy/x_dom_someip_rt_gen_uds/x_dom_someip_rt_cfg.cpp:36-41
```

Step 6: SDK触发proxy回调 → APP消费 (UDS进程)

```
【SDK】SvcItemInfo::set_origin_data() 内部:  
文件: plat-bf/cdd/comif/x_dom_someip_rt_com.h  
  
void set_origin_data(uint32_t id, const uint8_t *data, size_t len)  
{  
    memcpy(m_origin_data, data, len); // 缓存原始数据  
    m_rcv_count++;  
  
    // ★ SDK调用proxy回调 (函数指针)  
    if (m_rcv_func != nullptr)  
        m_rcv_func(id); // → SigIf_OnRecvData  
}
```

```
【Proxy】SigIf_OnRecvData(id)  
文件: proxy/x_dom_someip_rt_gen_uds/x_dom_someip_rt_sigif_get.cpp:31  
  
void SigIf_OnRecvData(uint32_t id)  
{  
    switch (id)  
    {  
        case SERVER_ID_..._RRSelfDownloadCmd_HAD_J0_RX:  
            if (SigIf_xxx_Cb != nullptr)  
            {  
                cpp_PS_SelfDLCmd_strt data;  
                // ★ Proxy调SDK反序列化读取  
                SigIf_xxx_Get(&data); // → Step 7  
                SigIf_xxx_Cb(&data, SigIf_xxx_Ctx); // 用户回调  
  
                break;  
            }  
            // ... 其他case  
    }
```

#### Step 7: APP/Proxy调用SDK反序列化

APP也可以主动读:

```
cpp_PS_SelfDLCmd_strt data;  
SigIf_xxx_Get(&data);
```

【Proxy】SigIf\_xxx\_Get(pData)

文件: proxy/x\_dom\_someip\_rt\_gen\_uds/x\_dom\_someip\_rt\_sigif\_get.cpp

```
return someip_recv_svc_info_xxx  
.get_usr_data_ext<cpp_PS_SelfDLCmd_strt>(  
    0, // offset  
    pData, // 输出  
    Deserialize // ← * Proxy提供的反序列化函数  
);
```

【SDK】SvcItemInfo::get\_usr\_data\_ext<>()

文件: plat-bf/cdd/comif/x\_dom\_someip\_rt\_com.h

```
template<typename T>  
int32_t get_usr_data_ext(uint32_t offset, T* pData,  
    bool (*deserialize)(...))  
{  
    // ← SDK调proxy的反序列化函数  
    return deserialize(m_origin_data + offset,  
        m_origin_data_size - offset, pData);  
}
```

【Proxy】Deserialize()

文件: proxy/x\_dom\_someip\_rt\_gen\_uds/x\_dom\_someip\_rt\_protocol\_0.cpp

```
bool Deserialize(const uint8_t* data, uint32_t len,  
    cpp_PS_SelfDLCmd_strt* out)  
{  
    out->field1 = read_uint32(data + 0);  
    out->field2 = read_uint8(data + 4);  
    // ... SOME/IP序列化格式 → C++结构体  
}
```

### 3.2 数据流出: UDS → COM → 外部ECU

#### Step 1: APP调用proxy Set

APP:

```
cpp_PS_SelfDLStatus_strt status = {...};  
SigIf_xxx_Set(&status);
```

【Proxy】SigIf\_xxx\_Set(pData)

文件: proxy/x\_dom\_someip\_rt\_gen\_uds/x\_dom\_someip\_rt\_sigif\_set.cpp

```
int32_t ret = someip_send_svc_info_xxx  
.set_usr_data_ext<>(0, pData, Serialize); // ← SDK序列化  
  
if (ret == 0)  
    someip_send_svc_info_xxx  
        .fwd_svc_via_dcms( // ← SDK通过DCMS发送  
            "/x_dom_someip_rt/app_to_com", false);
```

【SDK】SvcItemInfo::fwd\_svc\_via\_dcms()

→ SvcDataPackHdl打包  
→ dcms\_mcu\_topic\_send() → DCMS

#### Step 2: DCMS → COM

UDS进程 —DCMS topic "/x\_dom\_someip\_rt/app\_to\_com"—> COM进程

Step 3: COM SDK接收 → 查proxy send map → 发SOME/IP

```
【SDK】 svc_route_app_to_com_topic_cbk(data, len)
文件: plat-bf/cdd/x_dom_someip_rt/service_route.cpp:107

SvcDataUnPackHdl DataUnPackHdl(data, len);
while (1)
{
    DataUnPackHdl.get_next_data(item_id, item_data, item_len, ...);

    // * 查proxy的send map
    auto item_it = g_x_dom_someip_config
        .svc_route_send_svc_info_map_ptr->find(item_id);
    item_ptr = item_it->second;
    item_ptr->set_data(item_id, item_data, item_len);

    // * 通过SOME/IP发送到外部ECU
    item_ptr->fwd_svc_via_someip();
}

内部: fwd_svc_via_someip()
→ Cdd SomeIpTp Tx path (AUTOSAR SomeIpTp发送)
→ SoAd → Ethernet → 外部ECU
```

## 四、CAN 信号完整调用链 (COM → UDS)

Step 1: MCU CAN帧 → COM进程

MCU(R52核) —DCMS topic "mcu\_tx\_rx\_msgs"—> COM进程

```
【SDK】 msg_route_mcu_tx_rx_msgs_topic_cbk(data, len)
文件: plat-bf/cdd/x_dom_can_rt/msg_route_soc.cpp
→ 推入 mcu_can_msg_queue (ThreadSafeQueue)

【SDK】 x_dom_can_rt_main_func() [50ms定时]
→ msg_route_main_function()
→ msg_route_mcu_tx_rx_can_msgs_proc()
```

Step 2: SDK查proxy CanMsg路由表

```
【SDK】 msg_route_mcu_tx_rx_can_msgs_proc()
文件: plat-bf/cdd/x_dom_can_rt/msg_route_soc.cpp:137-169

// 构造查找key: {channel, can_id}
key_to_find = {can_msg.channel, can_msg.id};

// * SDK查proxy的map
auto it = msg_route_mcu_tx_rx_msgs_map.find(key_to_find);
can_msg_ptr = it->second;

// * SDK写入proxy定义的CanMsg缓存
can_msg_ptr->set_data(can_msg.data, can_msg.dlc, can_msg.status);

【Proxy】 msg_route_mcu_tx_rx_msgs_map
文件: proxy/x_dom_can_rt_gen_com/x_dom_can_rt_cfg.cpp:77

const map<CanMsgIdPair, CanMsg*> msg_route_mcu_tx_rx_msgs_map = {
    {{0, 0x118}, &mcu_tx_msg_VDC_1 default 0x118},
    {{0, 0xc0}, &mcu_rx_msg_ABS_1 default 0xc0},
    // ... 37个
};
```



```
【SDK】 CanMsg::set_data(data, len, status)
文件: plat-bf/cdd/msg_route_soc.h
{
    lock_guard<mutex> lock(mtx);
    memcpy(data_buf, data, len); // 缓存CAN帧原始字节
    m_recv_count++;
}
```

#### Step 3: SDK调proxy读取信号 → DCMS发送

```
【SDK】 sig_route_send_mcu_tx_rx_sigs_proc()
文件: plat-bf/cdd/x_dom_can_rt/sig_route_soc.cpp:25-31

#ifdef(XDOMCANRT_DJI_COM_SERVICE_EN) // COM域=1
    sig_route_send_mcu_tx_rx_sigs(); // * SDK调proxy函数
#endif

【Proxy】 sig_route_send_mcu_tx_rx_sigs()
文件: proxy/x_dom_can_rt/gen_com/x_dom_can_rt_cfg.cpp:645
{
    sig_route_send_mcu_tx_rx_sigs_dji_uds_service(); // 发给UDS
    sig_route_send_mcu_tx_rx_sigs_dji_application(); // 发给AD

    内部 (以dji_application为例):
    // * Proxy调SDK模板: 从CanMsg读信号物理值
    VDC_DecelerationReq_AEB_sighdl.signal_if_get(raw_value);

    // 打包为结构体
    send_struct.VDC_DecelerationReq_AEB = raw_value;
    // ... 60+个信号

    // 发送
    dcms_mcu_topic_send(topic_id, &send_struct, sizeof(send_struct));

【SDK】 SigHdl::signal_if_get(phy)
文件: plat-bf/cdd/sig_handle_soc.hpp
{
    can_msg_get_data(raw_bytes, sizeof(raw_bytes)); // 读CanMsg缓存
    raw = extract_bits(raw_bytes, start_bit, len, is_big_endian);
    phy = (phy_t)((float)raw * factor + offset); // 物理值转换
}
```

```
【Proxy】 SigHdl对象定义
文件: proxy/x_dom_can_rt/gen_com/x_dom_can_rt_cfg.cpp:131

SigHdl<float, uint16_t, float, float>
VDC_DecelerationReq_AEB_sighdl(
    0.01, 0.01, -10.21, -10.21, 0.2, // factor, offset
    64, 10, false, // start_bit=64, len=10, Intel格式
    mcu_tx_msg_VDC_1_default_0x118, // 引用的CanMsg
    debug_helper);
```

#### Step 4: COM内APP直接读信号 (同进程, 不走DCMS)

```
APP (COM进程内):
float decel;
SigIf_Get_VDC_DecelerationReq_AEB_VDC_1_Get(&decel);

【Proxy】 SigIf_Get_xxx(pData)
文件: proxy/x_dom_can_rt/gen_com/x_dom_can_rt_sigif.cpp
// * Proxy调SDK
return VDC_DecelerationReq_AEB_sighdl.signal_if_get(*pData);
// * SigHdl直接从CanMsg缓存读取+解码
```

## Step 5: UDS进程收CAN信号

COM → DCMS topic "/x\_dom\_can\_rt/read\_sigs/uds\_app" → UDS

**[SDK]** sig\_route\_mcu\_rx\_sigs\_topic\_cbk(data, len)

文件: plat-bf/cdd/x\_dom\_can\_rt/sig\_route\_soc.cpp:66

// \* SDK调proxy

return sig\_route\_update\_mcu\_tx\_rx\_sigs\_from\_com(data, len);

**[Proxy]** sig\_route\_update\_mcu\_tx\_rx\_sigs\_from\_com(data, len)

文件: proxy/x\_dom\_can\_rt\_gen\_uds/x\_dom\_can\_rt\_cfg.cpp:53

lock\_guard<mutex> lock(x\_dom\_can\_rt\_mtx);

// hash校验防止数据错乱

if (hash匹配 && 长度匹配)

update\_flag = true;

memcpy(本地缓存结构体, data, len); // 拷贝整个信号结构体

## 五、UDS 域的 danvince 诊断路径 (不进 comif, 不走 DCMS)

外部诊断仪 —DoIP (UDP 13400)→ UDS自己SoAd

AUTOSAR DoIP

AUTOSAR PduR

AUTOSAR Dcm

dji\_rte.c

DiagSwc\_Callback.c

(RTE接口注册)

(DID读写回调)

DiagnosticsService.c

Dcm\_Callout\_Stubs.c

(车型适配诊断逻辑)

(Dcm钩子: 会话切换、

安全访问、文件传输)

DiagProxy\_Com.cpp ← \* 可调comif SigIf 读跨域数据

(C-C++桥接)

└─ SigIf\_xxx\_Get() → comif SDK → SvcItemInfo缓存

└─ SigIf\_xxx\_Set() → comif SDK → DCMS → COM

\* 这条路径完全不经COM进程

\* DoIP + Dcm 是UDS专用的协议栈

\* DiagProxy可以同时使用danvince诊断数据和comif跨域数据

## 六、有无comif的核心对比

无comif (danvince)

有comif (新架构)

Cdd\_SomeIpTp.c行为

732个服务: 空函数体

114个服务: 调用

→ 数据进LdCom

x\_dom\_someip\_rt\_rx\_handle()

→ 数据进comif SDK

|               |                  |                              |
|---------------|------------------|------------------------------|
| UDS如何获取       | 没有标准方式！          | 自动生成全链路：                     |
| SOMEIP数据      | ① 人工写IPC代码       | JSON配置+工具生成+                 |
|               | ② 或从Com信号转       | Cdd_SomeIpTp + proxy路由表      |
|               | ③ 或在UDS建SomeIpTp | + SigIf接口 + 序列化              |
| 跨域转发代码        | 人工手写             | * 工具自动生成，零手写                 |
| UDS侧数据接口      | 无标准接口            | SigIf_xxx_Get(&typed_struct) |
|               |                  | → 类型安全的C++结构体                |
| 序列化/反序列化      | 人工写              | proxy/protocol_*.cpp 工具生成    |
| 路由配置          | 硬编码在代码里          | JSON配置文件 → 改JSON重新生成即可       |
| COM-UDS数据格式   | 不统一              | SvcDataPackHdl 统一二进制打包       |
| SDK/Proxy角色分工 | 不存在proxy层        | * SDK=运行时引擎                  |
|               |                  | * Proxy=JSON生成的配置+接口         |
| 新增跨域服务        | 改代码+改IPC+测试      | 改JSON → 重新生成 → 完成            |

## 七、平台仓SDK ↔ 车型适配仓Proxy 函数调用关系汇总

|                                                              |                                                                                                    |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| SDK 调用 Proxy (通过指针/外部声明)                                     |                                                                                                    |
|                                                              |                                                                                                    |
| SDK函数                                                        | 调用的Proxy数据/函数                                                                                      |
|                                                              |                                                                                                    |
| svc_route_rx_handle()<br>service_route.cpp:80                | → g_x_dom_someip_config.svc_route_rcv_svc_info_map_ptr<br>(proxy的 svc_route_rcv_svc_info_map)      |
| svc_route_cycle_pub()<br>service_route.cpp                   | → g_x_dom_someip_config.svc_route_rcv_svc_info_list_ptr<br>(proxy的 NodeRecvInfo vector)            |
| svc_route_immediately_pub()                                  | → 同上 + check_and_send_svc_via_dcms()                                                               |
| svc_route_app_to_com_topic_cbk()<br>service_route.cpp:107    | → g_x_dom_someip_config.svc_route_send_svc_info_map_ptr<br>(proxy的 send map)                       |
| svc_route_com_to_app_topic_cbk()<br>service_route.cpp:147    | → g_x_dom_someip_config.svc_route_rcv_svc_info_map_ptr<br>(proxy的 rcv map)                         |
| msg_route_mcu_tx_rx_can_msgs_proc()<br>msg_route_soc.cpp:137 | → msg_route_mcu_tx_rx_msgs_map (proxy)<br>→ CanMsg::set_data() (SDK自己的类方法)                         |
| sig_route_send_mcu_tx_rx_sigs_proc()<br>sig_route_soc.cpp:25 | → sig_route_send_mcu_tx_rx_sigs() (proxy外部函数)                                                      |
| sig_route_mcu_rx_sigs_topic_cbk()<br>sig_route_soc.cpp:66    | → sig_route_update_mcu_tx_rx_sigs_from_com() (proxy)                                               |
| SvcItemInfo::set_origin_data()<br>x_dom_someip_rt_com.h      | → m_rcv_func(id) → SigIf_OnRecvData() (proxy回调)                                                    |
| x_dom_someip_rt_init()<br>x_dom_someip_rt.cpp                | → g_x_dom_someip_config.svc_route_write_someip_init_ptr<br>→ svc_route_write_someip_init() (proxy) |

|                                                       |                                                                               |
|-------------------------------------------------------|-------------------------------------------------------------------------------|
| Proxy 调用 SDK (通过include SDK头文件)                       |                                                                               |
|                                                       |                                                                               |
| Proxy函数                                               | 调用的SDK类/方法                                                                    |
|                                                       |                                                                               |
| SigIf_xxx_Get(pData)<br>x_dom_someip_rt_sigif_get.cpp | → SvcItemInfo::get_usr_data_ext<>()<br>→ Deserialize() (proxy自己的函数, 通过函数指针传入) |
| SigIf_xxx_Set(pData)                                  | → SvcItemInfo::set_usr_data_ext<>()                                           |



```

x_dom_someip_rt_sigif_set.cpp      → Serialize() (proxy自己的函数)
                                   → SvcItemInfo::fwd_svc_via_dcms()

SigIf_Get_xxx_CanSignal(pData)     → SigHdl::signal_if_get()
x_dom_can_rt_sigif.cpp              (SDK模板, proxy实例化)

sig_route_send_mcu_tx_rx_sigs()     → SigHdl::signal_if_get()
x_dom_can_rt_cfg.cpp                (遍历所有proxy定义的SigHdl对象)
                                   → dcms_mcu_topic_send() (SDK/DCMS API)

x_dom_someip_rt_prod_cfg()          → x_dom_someip_rt_set() (SDK)
x_dom_someip_rt_cfg.cpp              将proxy的数据指针注入SDK全局结构体

svc_route_write_someip_init()       → dcms_adp_get_topic_id() (SDK/DCMS)
x_dom_someip_rt_cfg.cpp              → dcms_mcu_topic_setup_callback() (SDK/DCMS)

Cdd_SomeIpTp.c (114个comif回调)     → x_dom_someip_rt_rx_handle() (SDK)
(AUTOSAR适配层, 非proxy)            * SOME/IP收包后直接进comif

```

## 八、总结

### 最终架构图

外部ECU → SOMEIP → COM进程 → DCMS → UDS进程

|  |               |              |
|--|---------------|--------------|
|  | SOMEIP收包      | SOMEIP数据只能   |
|  | (唯一有SomeIpTp) | 从COM通过DCMS获取 |

外部诊断仪 → DoIP → UDS进程  
(UDS自己的DoIP端口)

MCU → CAN(DCMS) → COM进程 → DCMS → UDS进程

#### 库依赖:

COM = BSP + plat\_bf\_cdd.so(同一份) + proxy\_com + autosar\_com  
UDS = BSP + plat\_bf\_cdd.so(同一份) + proxy\_uds + autosar\_uds

#### 关键区别:

COM有SomeIpTp+Sd (SOMEIP协议栈) → 能收SOMEIP  
UDS有DoIP+Dcm (诊断协议栈) → 能收诊断仪, 但不能收SOMEIP

#### comif做的事情:

1. JSON配置 → 工具生成 Cdd\_SomeIpTp.c 的comif调用点
2. JSON配置 → 工具生成 proxy 路由表 + SigIf接口 + 序列化代码
3. SDK运行时引擎 根据proxy配置 自动完成 跨域路由+打包+转发

3. SDK运行时引擎 根据proxy配置 自动完成 跨域路由+打包+转发

#### comif不做的事情:

1. 不替代DCMS (仍然用它做IPC)
2. 不替代SomeIpTp (仍然需要它解析SOMEIP协议)
3. 不替代Dcm (诊断服务仍然是danvince AUTOSAR路径)